

ARCHITECTURE NOTE

CV Pipeline Talend

Why this project, how it works, what decisions were made, and why
For developers and technical recruiters.

Supplementary Document for the Construction Guide - Benjamin Cerland

Table of Contents

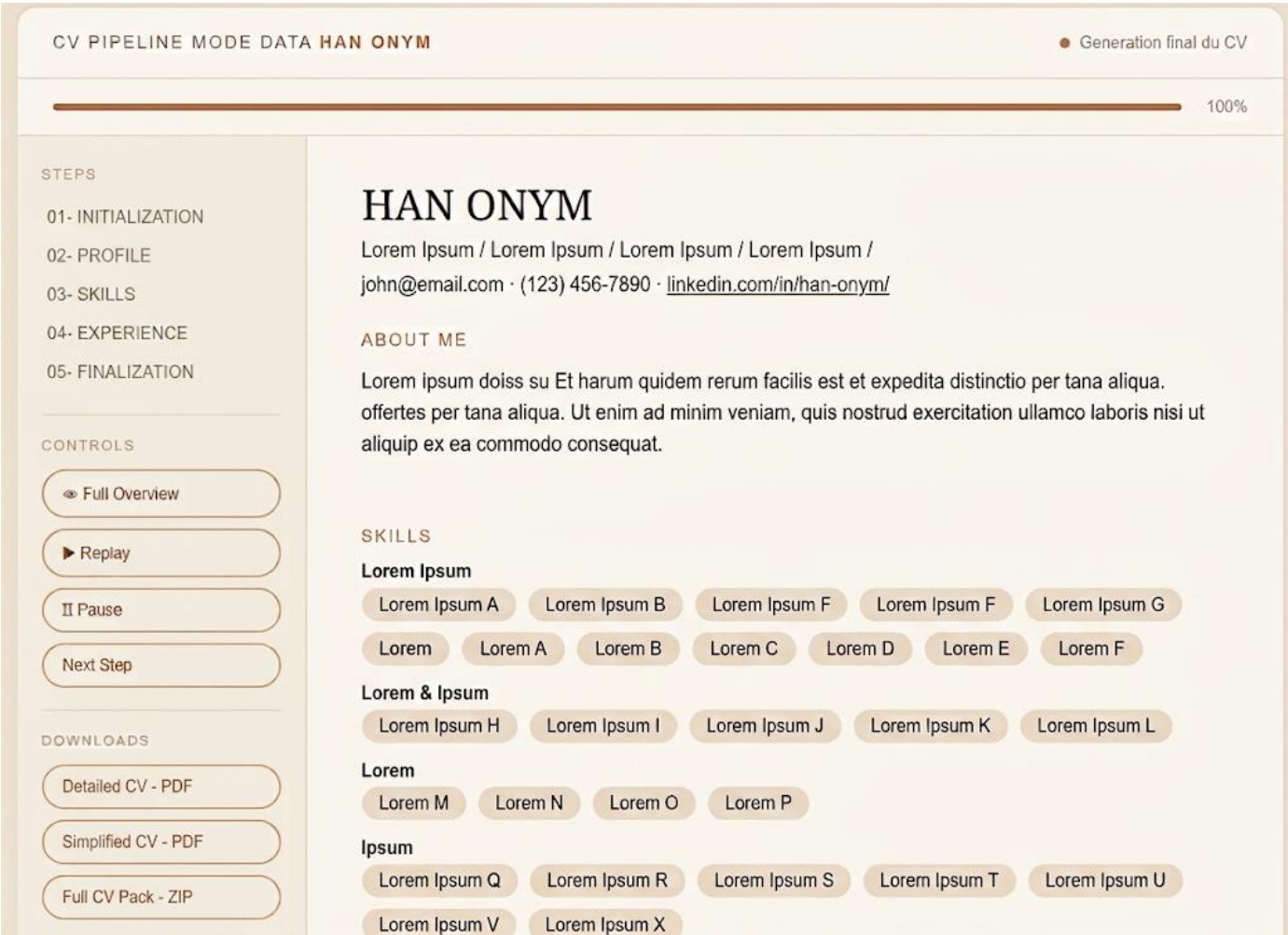
- 1. The problem this project solves
- 2. How it works
- 3. Architectural choices and why
- 4. What this project demonstrates technically
- 5. Known limitations and areas for improvement
- 6. In summary

1. The problem this project solves

A traditional resume is a static document: the recruiter reads it and has to take the listed skills at face value. For a Talend/Data role, there was a clear opportunity to do things differently: rather than writing “I know how to build data pipelines,” build one right in front of them—and have that pipeline generate the resume itself.

The goal is not novelty for novelty's sake: it is to demonstrate, in a real-world context, skills that would otherwise be merely listed: orchestration of processing, separation of data, logic, and presentation, state management, communication between two independent processes, and data structuring.

Nor is it a disposable gimmick: the project has gone through several real iterations (transitioning from a fetch/JSON (v1) architecture to a 100% serverless architecture, fixing configuration bugs, and revising design choices)—a normal development process, not a static demo that wouldn’t survive an initial test.



Actual screenshot of the interface — job completed (step 5 of 5), left column showing step tracking and controls

2. Operating Principle

The recruiter runs a Talend job. This job does not generate a complete HTML page at each step: it controls a web page that is already open, updating it on its progress via a shared file without ever going through a server or making a network request.



These are two independent processes (the Java/Talend job on one side, the browser on the other) that communicate solely through a file, which is loaded as an executable script rather than as data retrieved via a query. It is this specific choice, along with four others, that are detailed in the following section.

3. Architectural Choices, and Why

Why use a .js file loaded via the <script> tag instead of JSON via fetch() or a real WebSocket ?

There are three options for enabling communication between Talend and the browser: (1) `fetch()` from a JSON file, (2) a real server using WebSocket/SSE, (3) a reloaded `<script>` tag. The first option was tested first and ruled out: using `fetch()` on a local file (file://) triggers CORS restrictions in modern browsers, which required running a local server (Python, Node...), a dependency I specifically wanted to eliminate so that the project could be used by anyone without any installation.

The second approach would have solved the problem but at the cost of a much heavier dependency (a permanent server runtime) for a marginal benefit. The third approach—a .js file that runs and calls a browser function (the JSONP technique)—has no CORS restrictions, works just as well via file:// as it does when hosted online, and requires nothing more than a browser. It's a compromise: the format is no longer "pure" JSON, but a slightly less standard file. I decided that ease of use far outweighed the purity of the format.

```
//What Talend writes in RUNTIME/progress.js
onProgressUpdate({
  "step": 3,
  "total": 5,
  "status": "RUNNING",
  "message": "Processing skill dataset..."
});
```

Why use polling (every 500 ms) instead of a server push ?

Polling is the simplest solution, requiring no application server. The trade-off is a latency of 0 to 500 ms, which is imperceptible here since the business logic of the job runs at a rate of 3 seconds per step. A true push mechanism (Server-Sent Events, WebSocket) would be more elegant in theory but would require a permanent server runtime, which is outside the scope of a rendering solution that must work with a double-click, without requiring any installation.

Why do the Pause, Resume, and Step controls only work on the browser side ?

Because once a Talend job is launched, it runs autonomously and has no return channel to receive commands from the browser (which, again, would require a bidirectional server). The accepted trade-off: A client-side "Pause" only suspends the reading/display of the progress file, not the actual execution of the job, which continues to write to progress.js in the background. This is a pragmatic choice rather than a hidden limitation, and is documented as such in the development guide.

Why use tLoop + tJavaFlex instead of 5 separate subjobs ?

A single loop with a switch based on the iteration number is easier to maintain than a job with five parallel branches, and more closely resembles a classic ETL pattern (iterating over a set of steps) than a fixed sequence of components. The trade-off: the content logic (which message goes to which step) resides in the Java code rather than being defined by the job's visual structure—a choice that prioritizes the readability of the graph at the cost of some code readability.

Why split the data into four files (profile / skills / experience / education) instead of a single file ?

Each file corresponds to a section of the resume that changes at a different rate: work experience changes frequently, while education almost never does. Separating them reduces the risk of cascading syntax errors (a missing comma in experiences.js won't break the profile), makes each file shorter to review, and ultimately allows you to regenerate just one section without affecting the others. The cost: four <script> tags to load in order rather than just one, which is negligible.

4. What this project demonstrates from a technical standpoint

- **Process orchestration** : Correct use of Talend triggers (OnSubjobOk, Iterate, OnComponentOk) to build a controlled sequential flow.
- **State management** : Use of globalMap to pass a value between components within the same execution.
- **Portability** : Extraction of paths and parameters into context variables, rather than hard-coded paths.
- **Separation of concerns** : Three independent layers (data / processing / presentation), each replaceable without affecting the other two.
- **Interoperability without dependencies** : Communication between two heterogeneous runtimes (JVM and browser) without a server or third-party libraries, relying solely on native browser mechanisms (script loading).
- **Technical Compromises** : Every limitation (polling rather than push, client-side pausing only, JSONP format rather than pure JSON) is a deliberate and documented choice, not an oversight ;)

Techno 1

Tech 2

Techno 2

Techno 3

Techno 4

Techno 5

Tech 6

Techno 7

Techno 8

5. Known Limitations and Areas for Improvement

Current limit	Possible development
OS-Dependent File Paths (Windows / macOS / Linux)	Automatic OS detection in tSystem, or packaging with relative paths
No automatic PDF generation as output	Add a tFileOutputPDF at the end of the job (see the “Downloads” section of the design guide)
The job must be restarted manually from Talend Studio	Export as a standalone executable (.bat / .sh) for one-click execution without opening Studio
Latence de polling (jusqu'à 500 ms)	Polling latency (up to 500 ms)
Non-standard progress.js format (JSONP instead of JSON)	Acceptable for internal use within the project; should be avoided if the file were ever to be used by a third-party system

6. In Summary

This CV pipeline isn't just a visual gimmick: it's a genuine small-scale data project, built with a clear separation of layers, well-considered and documented architectural choices, and compromises that are explicitly explained rather than hidden.

This is a project I've wanted to undertake for some time, with the goal of offering a different way to present my background while concretely demonstrating my thought process and how I develop solutions. I'm particularly pleased with the result and the final presentation, and I hope you've enjoyed it as much as I enjoyed creating it.

It was designed to be read on two levels: by a non-technical recruiter, as an original, engaging, and personal presentation of my career path; and by a technical professional, as a concrete demonstration of how I approach integration challenges and evolve an architecture when necessary—particularly with the transition from an initial approach based on `fetch()` to the current, fully autonomous architecture.